SIMATS ENGINEERING

ERROR ANALYSIS TASK

Software Testing

Student Name	Bhavani Sankar
Register Number	192411238
Course Code	CSA3712
Course Outcome	CO2 – Design effective test cases using black-box and white-box testing techniques and control methods flow.
Assessment	CO2_AT3 – Error Analysis Task

Assignment Answers

1. Errors in Equivalence Class Partitioning and Boundary Value Analysis

Errors: The test designer has mixed valid and invalid values in the same equivalence class and has not tested values at the edges of input ranges. An equivalence class should contain inputs expected to behave similarly. Mixing classes can hide failures because one test result cannot represent both valid and invalid behaviour.

Root causes

- The input requirements and accepted range may not have been clearly identified.
- The tester may have created classes based on value appearance rather than expected system behaviour.
- Boundary values were ignored, even though errors often occur at the minimum and maximum limits.

Corrected approach and example

For an online shopping quantity field where the permitted quantity is 1–10, use these classes: invalid quantity below 1; valid quantity from 1 to 10; and invalid quantity above 10. Select representative values such as 0, 5, and 11. Apply Boundary Value Analysis with 0, 1, 2, 9, 10, and 11. If the actual field has a different permitted range, use that range instead.

Benefit: The corrected design separates expected valid and invalid behaviour and tests values immediately around limits. It is more likely to detect off-by-one errors, incorrect validation, and unexpected acceptance or rejection of quantities.

2. Evaluation of Random Testing in a Banking Application

Random testing alone is not sufficient to demonstrate high test coverage. Random values may repeatedly exercise ordinary transactions while missing limits, invalid values, and rare transaction states. A large number of tests does not automatically mean that important requirements or branches have been covered.

Errors and root causes

- No systematic selection of transaction amounts or boundary values.
- No evidence that every important requirement, decision, or transaction outcome was tested.
- Coverage is being claimed from the number of tests rather than measured against requirements or a defined coverage criterion.

Improved black-box techniques

- **Boundary Value Analysis:** test amounts just below, at, and just above minimum and maximum transaction limits.
- **Equivalence Class Partitioning:** test valid amounts, zero or negative amounts where disallowed, amounts over the limit, and malformed inputs.
- **Decision Table Testing:** combine account status, available balance, daily limit, and transaction amount to check approval or rejection rules.
- **State Transition Testing:** test outcomes such as initiated, approved, declined, and cancelled, where applicable.

Benefit: These techniques provide traceable and purposeful coverage of rules and limits. Random testing may still be used as an additional technique, but not as the only evidence of adequacy.

3. Incomplete Control Flow Graph and Poor Path Testing

A Control Flow Graph (CFG) must represent program statements as nodes and possible transfers of control as directed edges. The described CFG is incomplete because it omits decision nodes and loop edges. As a result, some branches and repeated executions cannot be represented or tested.

Correction

- Create a node for each relevant statement or basic block.

- Represent each condition as a decision node with separate true and false outgoing edges.
- Represent loop entry, loop body, loop exit, and the back edge to the condition.
- Check that every possible control transfer in the code is represented.

The exact independent paths depend on the program code, which is not included in the task sheet. For a simple example with one decision and one loop, representative paths are: (1) condition false and skip the loop/body; (2) condition true, execute the body once, then exit; and (3) execute the body more than once before exit, where repeated iteration is permitted.

Cyclomatic complexity can help estimate the number of linearly independent paths: V(G) = E – N + 2P, where E is the number of edges, N is the number of nodes, and P is the number of connected components (usually 1 for a single routine).

Benefit: A complete CFG reveals untested branches and loop behaviour. Independent-path tests improve structural coverage and help detect incorrect conditions, skipped statements, and loop-control defects.

4. Why 100% Statement Coverage Can Miss Logical Errors

Statement coverage confirms that each executable statement has run at least once. It does not prove that every decision outcome or combination of conditions has been tested. A test can execute a statement while exercising only one side of a conditional.

Example

For `if (balance >= amount)`, a test with sufficient balance may execute the successful transaction statement. If no test uses an amount greater than the balance, the rejection branch remains untested.

Recommended improvements

- **Branch coverage:** ensure every decision outcome (true and false) is executed.
- **Condition coverage:** make each Boolean condition evaluate to both true and false at least once.
- **Path coverage:** test important feasible paths, especially paths through nested decisions and error handling. Complete path coverage may be impractical when loops create many or infinite paths.

Benefit: Branch and condition testing can expose logical errors that statement coverage alone misses. The coverage criterion and results should be recorded clearly.

5. Correct Equivalence Classes and Boundaries for Age 0–100

The two classes “negative numbers” and “positive numbers” are inadequate. Positive numbers include valid ages from 1 to 100 as well as invalid ages above 100, while age 0 is not included in either class if zero is treated separately. The classes do not match the stated accepted range.

Correct partitions

- Invalid: age below 0.
- Valid: age from 0 through 100, inclusive.
- Invalid: age above 100.

Boundary Value Analysis

Test representative boundary values –1, 0, 1, 99, 100, and 101. Also test a typical valid age such as 25. The expected result is rejection for –1 and 101, and acceptance for 0, 1, 25, 99, and 100, assuming the specification allows ages 0–100 inclusive.

Benefit: The classes correctly distinguish permitted values from invalid values and the boundary tests can reveal incorrect comparison operators, such as using > instead of >=.

6. Condition Coverage for a Complex Boolean Decision

The supplied question is incomplete in the source document: it ends after “if(A && B” and does not provide the full condition, scenario, or stated testing mistake. Therefore, the exact defect and required test set cannot be determined from the provided material.

In general, condition coverage requires every individual Boolean condition to evaluate to both true and false at least once. For the illustrative expression `if (A && B)`, a minimal condition-coverage set can be: (A=true, B=true) and (A=false, B=true). These tests make A true and false, and B remains true, so this pair alone does not make B false. A complete condition-coverage set can use (true,true), (false,true), and (true,false). This set makes each condition evaluate to both outcomes. The overall decision outcome should also be checked if decision coverage is required.

Note: The example is illustrative, not a reconstruction of the missing question. The full question or code is needed to identify the exact expected tests.

7. Correct Decision Table for a Discount System

The current decision table is incomplete because it omits combinations of Premium Member and Coupon Applied. With two Boolean conditions, all four combinations must be considered unless the requirements explicitly make a combination impossible.

Illustrative exhaustive decision table

Rule	Premium member?	Coupon applied?	Expected discount handling
R1	No	No	Apply standard price; no member or coupon discount
R2	No	Yes	Apply coupon discount if valid
R3	Yes	No	Apply premium-member discount
R4	Yes	Yes	Apply the specified combined/priority rule

The task sheet does not specify discount percentages, coupon validity rules, or whether discounts are additive, sequential, or mutually exclusive. Those business rules must be confirmed before assigning an exact final price to R2–R4. Add tests for invalid or expired coupons if the system supports them.

Benefit: Exhaustive condition combinations prevent omissions and make the expected behaviour explicit, reducing inconsistent price calculations.

8. Correct Loop Testing Strategy

Testing only n=1 and n=max misses important loop behaviours. The loop may fail when it is not entered at all (n=0), and it may mishandle values just beyond the permitted maximum (n=max+1).

Simple loop tests

- 0 iterations: verify the loop body is skipped safely.
- 1 iteration: verify the first execution.
- 2 iterations: check repeated execution and loop update.
- A typical number of iterations.
- max–1, max, and max+1 iterations, where those values are meaningful and allowed by the specification.

Nested loops

For nested loops, begin with the innermost loop while holding outer loops at their minimum values. Then test each outer loop with the inner loops at representative values. Include zero, one, typical, and boundary iterations where feasible, and check that loop counters are initialized, updated, and terminated correctly.

Benefit: This approach detects off-by-one errors, incorrect termination, skipped execution, and interactions between nested loops. Tests should respect the specified input limits and avoid impractical exhaustive combinations.

9. Error Guessing in a Safety-Critical Braking System

Error Guessing relies on tester experience and intuition. Used alone for a safety-critical braking system, it is not systematic enough: it may omit required operating conditions, fail to document coverage, and overlook rare combinations that could have severe consequences.

Improved combined strategy

- **Boundary Value Analysis:** test values around specified braking thresholds, sensor ranges, response-time limits, and other defined boundaries. Exact values must come from system requirements.
- **State Transition Testing:** model relevant states such as normal operation, braking requested, braking active, fault detected, and safe/fallback mode, if these states exist in the design.
- Test valid and invalid transitions, including sensor faults, repeated brake requests, loss of input, and recovery, as defined by requirements.
- Retain error guessing as supplementary testing and add traceability, reviews, and risk-based prioritization.

Benefit: Structured techniques provide repeatable coverage of thresholds and state changes. For safety assurance, results should also be reviewed against applicable safety requirements and the project's required verification process.

10. Missing Else Branches in a Control Flow Graph

A CFG that omits the else branches of nested if statements is incomplete. It does not show every possible control transfer, so the graph can falsely suggest that all statements have been covered while error or alternative branches remain untested.

Correction

- Represent every if condition as a decision node with true and false outgoing edges.
- Create nodes for both the then and else blocks, including error-handling statements.
- Connect both branches to their correct successor or merge node, and represent nested decisions separately.
- Design tests so each decision takes both true and false outcomes. Include combinations that reach inner decisions and tests that follow error paths.

The exact CFG and independent paths cannot be drawn accurately without the module's source code. For a generic nested structure, paths should include: outer condition true and inner condition true; outer true and inner false; outer false; and any additional paths for further nested decisions. Where loops or early returns exist, include their edges and feasible paths too.

Use $V(G) = E - N + 2P$ to estimate the number of independent paths once the corrected CFG is constructed. Then map each selected path to one or more test cases.

Benefit: Representing all branches makes coverage reports meaningful and helps uncover missed error handling, incorrect nested conditions, and faulty control flow.